

Assignment - 1

Name: Rudra Dwivedi

Short Answer Question:

Q1. The primary difference is that a DBMS provides a structured, centralized way to manage, query, and secure data, while a traditional file system stores data in isolated, flat files. A DBMS offers data independence, concurrency control, built-in backup/recovery, and reduces data redundancy. Scenario: A banking system processing financial transactions. A DBMS is the better choice because it guarantees ACID (Atomicity, Consistency, Isolation, Durability) properties. If two users try to withdraw money from the same account simultaneously, the DBMS enforces concurrency control to prevent conflicting updates. A traditional file system would risk data corruption or inconsistent balances in this scenario.

Q2. A strong entity can exist independently and has its own unique primary key. A weak entity cannot exist without its parent (the strong entity) and lacks sufficient attributes to form a primary key on its own. The primary key of a weak entity is formed by combining the primary key of its associated strong entity with its own partial key (or discriminator). The relationship that connects the weak entity to the strong entity is called an identifying relationship.

Q3. It should be modeled as a multivalued attribute. Justification: A simple attribute can only hold a single atomic value, and a composite attribute is used to break a concept into sub-parts (like an address into street, city, zip). Because a single student entity can possess several independent instances of a phone number (e.g., home, mobile, work), a multivalued attribute perfectly represents this one-to-many trait without requiring database structural changes if a student adds a third or fourth number.

Q4. A super key is any set of attributes that uniquely identifies a row in a table. A candidate key is a minimal super key; it is a super key that contains no unnecessary or redundant attributes. Every candidate key is a super key, but not every super key is a candidate key. For the relation Student(roll_no, email, name, branch):

- Candidate keys: roll_no, email
- Super keys (that are not candidate keys): {roll_no, name}, {email, branch} (or {roll_no, email, name})

Q5. a) Referential Integrity Constraint (or Foreign Key Constraint).

b) It prevents "orphan records" and maintains logical consistency between related tables, ensuring that a record cannot reference a non-existent entity.

c) The DBMS should reject the insert operation and throw an error.

Q6. A) A junction table (also known as an associative or bridging table) must be created.

B) Its composite primary key consists of the foreign keys referencing the primary keys of

the participating entity tables.

C) Relational databases require First Normal Form (1NF), meaning each cell must hold a single, atomic value. Placing the relationship in an existing column would require storing a list of multiple values (violating 1NF) or creating massive data redundancy by duplicating entire rows for every pairing.

Long Answer Question

Part A: Entities and Key Attributes:

- Customer: CustomerID (Key)
- Restaurant: RestaurantID (Key)
- Order: OrderID (Key)
- MenuItem: ItemID (Key)

Relationships and Cardinality:

- Customer Places Order (1:N): A customer can place multiple orders over time, but a specific order is placed by exactly one customer.
- Restaurant Receives Order (1:N): Each order goes to exactly one restaurant, but a restaurant receives multiple orders.
- Order Contains MenuItem (M:N): An order can consist of multiple menu items, and a specific menu item can be ordered in multiple different orders.
- Restaurant Offers MenuItem (1:N): A restaurant offers multiple items on its menu, and we assume a specific menu item record belongs to one specific restaurant.

Part B:

- One multivalued attribute: Phone_Number for the Customer entity, because, a single customer might have a mobile number and a landline, meaning the system needs to store multiple values for the same attribute.
- Address for the Customer (or Restaurant) entity, since an address can be logically broken down into smaller, meaningful sub-attributes such as Street, City, State, and Zip_Code.

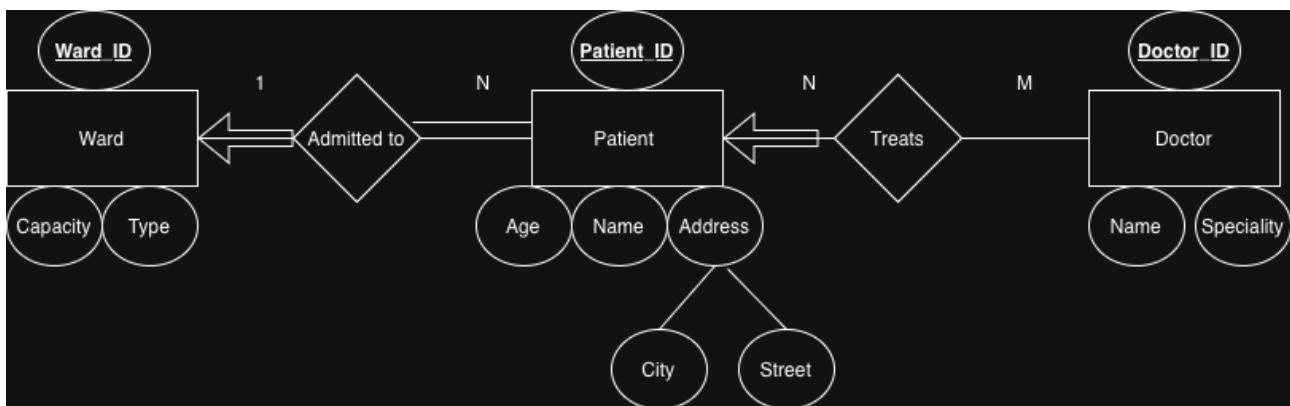
Part C:

For the M:N relationship between Order and Menu Item:

Order_MenuItem(OrderID, ItemID, quantity)

- Primary Key: (OrderID, ItemID)
- Foreign Keys:
 - OrderID referencing Order(OrderID)
 - ItemID referencing MenuItem(ItemID)
- Additional Attributes: quantity (to represent how many of that specific item were ordered).

Part C: ER diagram



In the hospital scenario, the Doctor-Patient relationship is M:N because a single doctor treats multiple patients, and a single patient may require treatment from multiple doctors during their stay (e.g., a primary physician and a specialist). This M:N relationship creates an associative entity, which could be named 'Treatment' or 'Consultation'. The composite primary key for this associative entity would consist of the primary keys of both participating entities: (DoctorID, PatientID).

Regarding participation constraints, the Patient entity has total participation in the 'Admitted_To' relationship with the Ward. This is because every patient in the context of this system must be officially assigned to a specific ward during their admission. Conversely, the Ward entity has partial participation; a newly opened or recently cleared ward might currently house zero patients.

For an attribute design decision, I modeled the Patient's 'Address' as a composite attribute (broken down into Street, City, and State). This decision was made because addresses are frequently searched or filtered to analyze patient demographics by region. Storing it as a composite attribute ensures the database can easily query by these individual geographic components.